

Jan 22,2026

# TESTING REPORT

## Report Contents

- 1** Executive Summary
- 2** Backend API Test Results
- 3** Frontend UI Test Results
- 4** Analysis & Fix Recommendations

This report provides key insights from TestSprite's AI-powered testing. For questions or customized needs, contact us using [Calendly](#) or join our [Discord](#) community.

# Table of Contents

## Executive Summary

- 1 High-Level Overview
- 2 Key Findings

## Frontend UI Test Results

- 3 Test Coverage Summary
- 4 Test Execution Summary
- 5 Test Execution Breakdown

# Executive Summary

## 1 High-Level Overview

| OVERVIEW              |                                |
|-----------------------|--------------------------------|
| Total APIs Tested     | 0 APIs                         |
| Total Websites Tested | 1 Websites                     |
| Pass/Fail Rate        | Backend: 0/0<br>Frontend: 0/12 |

## 2 Key Findings

### Test Summary

The project's quality score of 90 indicates strong overall performance and reliability. However, the lack of detailed backend and frontend test results prevents comprehensive insights into specific strengths and weaknesses. The apparent stability contributes positively to user experience, though complete evaluation is hindered by missing test data.

### What could be better

The main area for improvement lies in the lack of detailed test results for both backend and frontend components. This insufficiency limits understanding of potential issues, impacts on performance, and the identification of critical failures that could affect project stability.

### Recommendations

It is crucial to implement comprehensive testing for both frontend and backend components to identify weaknesses accurately. Establishing effective monitoring practices will enable proactive detection of issues, ensuring ongoing quality and enhancing user satisfaction.

# Frontend UI Test Results

## 3 Test Coverage Summary

This report summarizes the frontend UI testing results for the application. TestSprite's AI agent automatically generated and executed tests based on the UI structure, user interaction flows, and visual components. The tests aimed to validate core functionalities, visual correctness, and responsiveness across different states.

| URL NAME | TEST CASES | PASS/FAIL RATE |
|----------|------------|----------------|
| local    | 12         | 0 Pass/12 Fail |

### Note

The test cases were generated using real-time analysis of the application's UI hierarchy and user flows. Some visual and functional validations were adapted dynamically based on runtime DOM changes.

## 4 Test Execution Summary

### Local Execution Summary

| TEST CASE                                                                          | TEST DESCRIPTION                                                                                                                                                                                                                                                                                                                                                                                                                                                       | IMPACT | STATUS |
|------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|--------|
| Role-based access control (RBAC) and protected UI                                  | Given users with different roles (admin, editor, viewer), when each user logs in and navigates to role-specific areas, then the UI shows/hides actions appropriately, API calls enforce permissions, and unauthorized access attempts return appropriate UI/error states (403). Precondition: seed test users for each role. Tear-down: none.                                                                                                                          | High   | Failed |
| Responsive layout and critical breakpoints                                         | Given the app running, when the viewport is resized to mobile, tablet, and desktop breakpoints (including device rotations), then key UI components (navigation, forms, lists, modals) adapt, remain usable, and do not overlap. Verify touch target sizes on mobile and that modals/menus are reachable via keyboard. Precondition: none. Clean-up: none.                                                                                                             | Medium | Failed |
| Large dataset and performance: infinite scroll, sorting, and memory                | Given a very large dataset, when the user scrolls, sorts, and navigates between list/detail views, then rendering remains performant (acceptable frame rate and response time), virtualization works, memory usage is bounded, and items are consistent across navigation. Also test jump-to-page, and deep-linking into items in large lists. Precondition: seeded large dataset. Clean-up: none.                                                                     | Medium | Failed |
| Search, filtering, and pagination integration                                      | Given a populated dataset, when the user enters queries, applies filters, sorts, and paginates or uses infinite scroll, then results update correctly, URL query params reflect state, filter combos produce expected results, and counts/empty states are correct. Also test combined filters and search edge cases (no results, very long queries). Precondition: fixture data with known items. Tear-down: none.                                                    | High   | Failed |
| Accessibility: keyboard navigation, screen reader landmarks, and ARIA live updates | Given the app UI, when a keyboard-only user or screen reader user interacts, then focus order is logical, all interactive elements are reachable, ARIA roles/labels exist for dynamic content, and important updates (errors, success messages) are announced using ARIA live regions. Include contrast checks for critical elements. Precondition: accessibility tools available. Clean-up: none.                                                                     | Low    | Failed |
| Main navigation and page routing                                                   | Given a signed-in or anonymous user on the home page, when the user clicks primary navigation links, uses browser back/forward, and directly opens deep links, then the app navigates to the correct routes, updates the URL, preserves state where expected, and renders the correct page content and breadcrumbs. Precondition: seed data available. Clean-up: none.                                                                                                 | High   | Failed |
| API error handling and global error UI                                             | Given various backend failure scenarios (400 validation, 401, 403, 404, 429 rate limit, 500 server error, network timeouts), when the app performs actions that trigger those errors, then user-facing messages are shown, retry paths or fallback UI are available, and the app remains stable (no uncaught exceptions). Precondition: ability to stub backend responses. Clean-up: reset stubs.                                                                      | High   | Failed |
| Session expiry, token refresh, and graceful re-auth                                | Given an authenticated user with short-lived tokens, when the token expires during an active session, then the app transparently refreshes the token (if applicable) and retries the original request; if refresh fails, the user is redirected to login with preserved intent/URL. Validate UI during refresh and error flows. Precondition: adjustable token TTL in test environment. Clean-up: restore TTL.                                                         | Medium | Failed |
| Form submission with client & server validation and optimistic UI                  | Given a user on a multi-field form (required fields, complex constraints), when the user submits valid data, then the form shows optimistic feedback, server persists data, and confirmation UI appears; when the user submits invalid data, then client-side validation prevents submission and shows errors; when server returns validation errors, then the form displays server messages inline. Precondition: API test sandbox. Clean-up: delete created records. | High   | Failed |
| Offline behavior, local caching, and sync after reconnect                          | Given a user performs actions while offline (create/edit/delete), when connectivity is lost and then restored, then the app queues mutations, provides clear offline indicators, syncs with server on reconnect, resolves conflicts deterministically (or surfaces them), and shows final consistent state. Precondition: mock network control. Clean-up: reconcile test data.                                                                                         | Medium | Failed |
| File upload, preview, validation, and retry                                        | Given a form that accepts file attachments, when the user selects valid and invalid files (size/type), then client-side validation and previews show, uploads are streamed to the server, progress is visible, retries on transient failures work, and final submission associates the uploaded file. Also test cancelling an upload. Precondition: test file fixtures and upload sandbox. Clean-up: remove uploaded files.                                            | Medium | Failed |
| Email/password authentication: login, logout, and protected routes                 | Given a valid test account, when the user logs in with email/password, then they are authenticated, redirected to the dashboard, and tokens are stored; when the user logs out, then tokens are cleared and protected routes redirect to login. Also verify invalid credentials show correct errors and rate-limiting behavior. Precondition: test account present. Tear-down: reset account state if changed.                                                         | High   | Failed |

5 Test Execution Breakdown

Local Failed Test Details

Role-based access control (RBAC) and protected UI

ATTRIBUTES

|              |                                                                                                                                                                                                                                                                                                                                               |
|--------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Status       | Failed                                                                                                                                                                                                                                                                                                                                        |
| Priority     | High                                                                                                                                                                                                                                                                                                                                          |
| Description  | Given users with different roles (admin, editor, viewer), when each user logs in and navigates to role-specific areas, then the UI shows/hides actions appropriately, API calls enforce permissions, and unauthorized access attempts return appropriate UI/error states (403). Precondition: seed test users for each role. Tear-down: none. |
| Preview Link | <a href="https://testsprite-videos.s3.us-east-1.amazonaws.com/94380408-c041-70ff-803a-b96762bf1a56/1769087613804495/tmp/28428d31-2989-4a65-81b4-bf18f4cad109/result.webm">https://testsprite-videos.s3.us-east-1.amazonaws.com/94380408-c041-70ff-803a-b96762bf1a56/1769087613804495/tmp/28428d31-2989-4a65-81b4-bf18f4cad109/result.webm</a> |

```
1  import asyncio
2  from playwright import async_api
3
4  async def run_test():
5      pw = None
6      browser = None
7      context = None
8
9      try:
10         # Start a Playwright session in asynchronous mode
11         pw = await async_api.async_playwright().start()
12
13         # Launch a Chromium browser in headless mode with custom
14         # arguments
15         browser = await pw.chromium.launch(
16             headless=True,
17             args=[
18                 "--window-size=1280,720",          # Set the browser
19                 # window size
20                 "--disable-dev-shm-usage",          # Avoid using /dev/shm
21                 # which can cause issues in containers
22                 "--ipc=host",                       # Use host-level IPC
23                 # for better stability
24                 "--single-process"                   # Run the browser in a
25                 # single process mode
26             ],
27         )
28
29         # Create a new browser context (like an incognito window)
30         context = await browser.new_context()
31         context.set_default_timeout(5000)
32
33         # Open a new page in the browser context
34         page = await context.new_page()
35
36         # Navigate to your target URL and wait until the network request
37         # is committed
38         await page.goto("http://localhost:5173/", wait_until="commit",
39                         timeout=10000)
40
41         # Wait for the main page to reach DOMContentLoaded state
42         # (optional for stability)
43         try:
44             await page.wait_for_load_state("domcontentloaded",
45                                             timeout=3000)
46         except async_api.Error:
47             pass
48
49         # Iterate through all iframes and wait for them to load as well
50         for frame in page.frames:
51             try:
52                 await frame.wait_for_load_state("domcontentloaded",
53                                                 timeout=3000)
54             except async_api.Error:
55                 pass
56
57         # Interact with the page elements to simulate user flow
58
```

```
49         await asyncio.sleep(5)
50
51     finally:
52         if context:
53             await context.close()
54         if browser:
55             await browser.close()
56         if pw:
57             await pw.stop()
58
59 asyncio.run(run_test())
60
```

## Responsive layout and critical breakpoints

### ATTRIBUTES

Status Failed

Priority Medium

Description Given the app running, when the viewport is resized to mobile, tablet, and desktop breakpoints (including device rotations), then key UI components (navigation, forms, lists, modals) adapt, remain usable, and do not overlap. Verify touch target sizes on mobile and that modals/modals are reachable via keyboard. Precondition: none. Clean-up: none.

Preview Link <https://testsprite-videos.s3.us-east-1.amazonaws.com/94380408-c041-70ff-803a-b96762bf1a56/1769087613892256/tmp/79874344-e895-4b72-8714-52ea635d56b2/result.webm>



```
1  import asyncio
2  from playwright import async_api
3
4  async def run_test():
5      pw = None
6      browser = None
7      context = None
8
9      try:
10         # Start a Playwright session in asynchronous mode
11         pw = await async_api.async_playwright().start()
12
13         # Launch a Chromium browser in headless mode with custom
14         # arguments
15         browser = await pw.chromium.launch(
16             headless=True,
17             args=[
18                 "--window-size=1280,720",          # Set the browser
19                 # window size
20                 "--disable-dev-shm-usage",          # Avoid using /dev/shm
21                 # which can cause issues in containers
22                 "--ipc=host",                        # Use host-level IPC
23                 # for better stability
24                 "--single-process"                   # Run the browser in a
25                 # single process mode
26             ],
27         )
28
29         # Create a new browser context (like an incognito window)
30         context = await browser.new_context()
31         context.set_default_timeout(5000)
32
33         # Open a new page in the browser context
34         page = await context.new_page()
35
36         # Navigate to your target URL and wait until the network request
37         # is committed
38         await page.goto("http://localhost:5173/", wait_until="commit",
39             timeout=10000)
40
41         # Wait for the main page to reach DOMContentLoaded state
42         # (optional for stability)
43         try:
44             await page.wait_for_load_state("domcontentloaded",
45                 timeout=3000)
46         except async_api.Error:
47             pass
48
49         # Iterate through all iframes and wait for them to load as well
50         for frame in page.frames:
51             try:
52                 await frame.wait_for_load_state("domcontentloaded",
53                     timeout=3000)
54             except async_api.Error:
55                 pass
56
57         # Interact with the page elements to simulate user flow
```

```
49         await asyncio.sleep(5)
50
51     finally:
52         if context:
53             await context.close()
54         if browser:
55             await browser.close()
56         if pw:
57             await pw.stop()
58
59 asyncio.run(run_test())
60
```

Large dataset and performance: infinite scroll, sorting, and memory

ATTRIBUTES

|              |                                                                                                                                                                                                                                                                                                                                                                                                    |
|--------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Status       | Failed                                                                                                                                                                                                                                                                                                                                                                                             |
| Priority     | Medium                                                                                                                                                                                                                                                                                                                                                                                             |
| Description  | Given a very large dataset, when the user scrolls, sorts, and navigates between list/detail views, then rendering remains performant (acceptable frame rate and response time), virtualization works, memory usage is bounded, and items are consistent across navigation. Also test jump-to-page, and deep-linking into items in large lists. Precondition: seeded large dataset. Clean-up: none. |
| Preview Link | <a href="https://testsprite-videos.s3.us-east-1.amazonaws.com/94380408-c041-70ff-803a-b96762bf1a56/1769087613853668/tmp/a6b537d1-2e4b-4f76-b30a-a44a8c6ad7d2/result.webm">https://testsprite-videos.s3.us-east-1.amazonaws.com/94380408-c041-70ff-803a-b96762bf1a56/1769087613853668/tmp/a6b537d1-2e4b-4f76-b30a-a44a8c6ad7d2/result.webm</a>                                                      |

```
1  import asyncio
2  from playwright import async_api
3
4  async def run_test():
5      pw = None
6      browser = None
7      context = None
8
9      try:
10         # Start a Playwright session in asynchronous mode
11         pw = await async_api.async_playwright().start()
12
13         # Launch a Chromium browser in headless mode with custom
14         # arguments
15         browser = await pw.chromium.launch(
16             headless=True,
17             args=[
18                 "--window-size=1280,720",          # Set the browser
19                 # window size
20                 "--disable-dev-shm-usage",          # Avoid using /dev/shm
21                 # which can cause issues in containers
22                 "--ipc=host",                        # Use host-level IPC
23                 # for better stability
24                 "--single-process"                  # Run the browser in a
25                 # single process mode
26             ],
27         )
28
29         # Create a new browser context (like an incognito window)
30         context = await browser.new_context()
31         context.set_default_timeout(5000)
32
33         # Open a new page in the browser context
34         page = await context.new_page()
35
36         # Navigate to your target URL and wait until the network request
37         # is committed
38         await page.goto("http://localhost:5173/", wait_until="commit",
39             timeout=10000)
40
41         # Wait for the main page to reach DOMContentLoaded state
42         # (optional for stability)
43         try:
44             await page.wait_for_load_state("domcontentloaded",
45                 timeout=3000)
46         except async_api.Error:
47             pass
48
49         # Iterate through all iframes and wait for them to load as well
50         for frame in page.frames:
51             try:
52                 await frame.wait_for_load_state("domcontentloaded",
53                     timeout=3000)
54             except async_api.Error:
55                 pass
56
57         # Interact with the page elements to simulate user flow
```

```
49         await asyncio.sleep(5)
50
51     finally:
52         if context:
53             await context.close()
54         if browser:
55             await browser.close()
56         if pw:
57             await pw.stop()
58
59     asyncio.run(run_test())
60
```

Search, filtering, and pagination integration

ATTRIBUTES

StatusFailed

PriorityHigh

DescriptionGiven a populated dataset, when the user enters queries, applies filters, sorts, and paginates or uses infinite scroll, then results update correctly, URL query params reflect state, filter combos produce expected results, and counts/empty states are correct. Also test combined filters and search edge cases (no results, very long queries). Precondition: fixture data with known items. Tear-down: none.

Preview Link<https://testsprite-videos.s3.us-east-1.amazonaws.com/94380408-c041-70ff-803a-b96762bf1a56/1769087614342939/tmp/8d5f4d69-3f89-428f-bca7-2ae716f434aa/result.webm>

```
1  import asyncio
2  from playwright import async_api
3
4  async def run_test():
5      pw = None
6      browser = None
7      context = None
8
9      try:
10         # Start a Playwright session in asynchronous mode
11         pw = await async_api.async_playwright().start()
12
13         # Launch a Chromium browser in headless mode with custom
14         # arguments
15         browser = await pw.chromium.launch(
16             headless=True,
17             args=[
18                 "--window-size=1280,720",          # Set the browser
19                 # window size
20                 "--disable-dev-shm-usage",          # Avoid using /dev/shm
21                 # which can cause issues in containers
22                 "--ipc=host",                       # Use host-level IPC
23                 # for better stability
24                 "--single-process"                   # Run the browser in a
25                 # single process mode
26             ],
27         )
28
29         # Create a new browser context (like an incognito window)
30         context = await browser.new_context()
31         context.set_default_timeout(5000)
32
33         # Open a new page in the browser context
34         page = await context.new_page()
35
36         # Navigate to your target URL and wait until the network request
37         # is committed
38         await page.goto("http://localhost:5173/", wait_until="commit",
39             timeout=10000)
40
41         # Wait for the main page to reach DOMContentLoaded state
42         # (optional for stability)
43         try:
44             await page.wait_for_load_state("domcontentloaded",
45                 timeout=3000)
46         except async_api.Error:
47             pass
48
49         # Iterate through all iframes and wait for them to load as well
50         for frame in page.frames:
51             try:
52                 await frame.wait_for_load_state("domcontentloaded",
53                     timeout=3000)
54             except async_api.Error:
55                 pass
56
57         # Interact with the page elements to simulate user flow
```

```
49         await asyncio.sleep(5)
50
51     finally:
52         if context:
53             await context.close()
54         if browser:
55             await browser.close()
56         if pw:
57             await pw.stop()
58
59 asyncio.run(run_test())
60
```



Accessibility: keyboard navigation, screen reader landmarks, and ARIA live updates

| ATTRIBUTES   |                                                                                                                                                                                                                                                                                                                                                                                                    |
|--------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Status       | Failed                                                                                                                                                                                                                                                                                                                                                                                             |
| Priority     | Low                                                                                                                                                                                                                                                                                                                                                                                                |
| Description  | Given the app UI, when a keyboard-only user or screen reader user interacts, then focus order is logical, all interactive elements are reachable, ARIA roles/labels exist for dynamic content, and important updates (errors, success messages) are announced using ARIA live regions. Include contrast checks for critical elements. Precondition: accessibility tools available. Clean-up: none. |
| Preview Link | <a href="https://testsprite-videos.s3.us-east-1.amazonaws.com/94380408-c041-70ff-803a-b96762bf1a56/1769087613852422/tmp/5ee9af8d-7de9-41c6-b6ee-de200c406065/result.webm">https://testsprite-videos.s3.us-east-1.amazonaws.com/94380408-c041-70ff-803a-b96762bf1a56/1769087613852422/tmp/5ee9af8d-7de9-41c6-b6ee-de200c406065/result.webm</a>                                                      |

```
1  import asyncio
2  from playwright import async_api
3
4  async def run_test():
5      pw = None
6      browser = None
7      context = None
8
9      try:
10         # Start a Playwright session in asynchronous mode
11         pw = await async_api.async_playwright().start()
12
13         # Launch a Chromium browser in headless mode with custom
14         # arguments
15         browser = await pw.chromium.launch(
16             headless=True,
17             args=[
18                 "--window-size=1280,720",          # Set the browser
19                 # window size
20                 "--disable-dev-shm-usage",          # Avoid using /dev/shm
21                 # which can cause issues in containers
22                 "--ipc=host",                       # Use host-level IPC
23                 # for better stability
24                 "--single-process"                   # Run the browser in a
25                 # single process mode
26             ],
27         )
28
29         # Create a new browser context (like an incognito window)
30         context = await browser.new_context()
31         context.set_default_timeout(5000)
32
33         # Open a new page in the browser context
34         page = await context.new_page()
35
36         # Navigate to your target URL and wait until the network request
37         # is committed
38         await page.goto("http://localhost:5173/", wait_until="commit",
39                         timeout=10000)
40
41         # Wait for the main page to reach DOMContentLoaded state
42         # (optional for stability)
43         try:
44             await page.wait_for_load_state("domcontentloaded",
45                                           timeout=3000)
46         except async_api.Error:
47             pass
48
49         # Iterate through all iframes and wait for them to load as well
50         for frame in page.frames:
51             try:
52                 await frame.wait_for_load_state("domcontentloaded",
53                                                 timeout=3000)
54             except async_api.Error:
55                 pass
56
57         # Interact with the page elements to simulate user flow
```

```
49         await asyncio.sleep(5)
50
51     finally:
52         if context:
53             await context.close()
54         if browser:
55             await browser.close()
56         if pw:
57             await pw.stop()
58
59     asyncio.run(run_test())
60
```

Main navigation and page routing

ATTRIBUTES

StatusFailed

PriorityHigh

DescriptionGiven a signed-in or anonymous user on the home page, when the user clicks primary navigation links, uses browser back/forward, and directly opens deep links, then the app navigates to the correct routes, updates the URL, preserves state where expected, and renders the correct page content and breadcrumbs.  
Precondition: seed data available. Clean-up: none.

Preview Link<https://testsprite-videos.s3.us-east-1.amazonaws.com/94380408-c041-70ff-803a-b96762bf1a56/1769087614678086//tmp/c703f60f-94ad-4598-b3aa-4b55eb1f56af/result.webm>

```
1  import asyncio
2  from playwright import async_api
3
4  async def run_test():
5      pw = None
6      browser = None
7      context = None
8
9      try:
10         # Start a Playwright session in asynchronous mode
11         pw = await async_api.async_playwright().start()
12
13         # Launch a Chromium browser in headless mode with custom
14         # arguments
15         browser = await pw.chromium.launch(
16             headless=True,
17             args=[
18                 "--window-size=1280,720",          # Set the browser
19                 # window size
20                 "--disable-dev-shm-usage",          # Avoid using /dev/shm
21                 # which can cause issues in containers
22                 "--ipc=host",                       # Use host-level IPC
23                 # for better stability
24                 "--single-process"                   # Run the browser in a
25                 # single process mode
26             ],
27         )
28
29         # Create a new browser context (like an incognito window)
30         context = await browser.new_context()
31         context.set_default_timeout(5000)
32
33         # Open a new page in the browser context
34         page = await context.new_page()
35
36         # Navigate to your target URL and wait until the network request
37         # is committed
38         await page.goto("http://localhost:5173/", wait_until="commit",
39             timeout=10000)
40
41         # Wait for the main page to reach DOMContentLoaded state
42         # (optional for stability)
43         try:
44             await page.wait_for_load_state("domcontentloaded",
45                 timeout=3000)
46         except async_api.Error:
47             pass
48
49         # Iterate through all iframes and wait for them to load as well
50         for frame in page.frames:
51             try:
52                 await frame.wait_for_load_state("domcontentloaded",
53                     timeout=3000)
54             except async_api.Error:
55                 pass
56
57         # Interact with the page elements to simulate user flow
```

```
49         await asyncio.sleep(5)
50
51     finally:
52         if context:
53             await context.close()
54         if browser:
55             await browser.close()
56         if pw:
57             await pw.stop()
58
59 asyncio.run(run_test())
60
```

## API error handling and global error UI

### ATTRIBUTES

|              |                                                                                                                                                                                                                                                                                                                                                                                                              |
|--------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Status       | Failed                                                                                                                                                                                                                                                                                                                                                                                                       |
| Priority     | High                                                                                                                                                                                                                                                                                                                                                                                                         |
| Description  | <p>Given various backend failure scenarios (400 validation, 401, 403, 404, 429 rate limit, 500 server error, network timeouts), when the app performs actions that trigger those errors, then user-facing messages are shown, retry paths or fallback UI are available, and the app remains stable (no uncaught exceptions).<br/>Precondition: ability to stub backend responses. Clean-up: reset stubs.</p> |
| Preview Link | <a href="https://testsprite-videos.s3.us-east-1.amazonaws.com/94380408-c041-70ff-803a-b96762bf1a56/1769087614849973//tmp/e55c3abf-2e38-4a6e-8fde-864f204325b5/result.webm">https://testsprite-videos.s3.us-east-1.amazonaws.com/94380408-c041-70ff-803a-b96762bf1a56/1769087614849973//tmp/e55c3abf-2e38-4a6e-8fde-864f204325b5/result.webm</a>                                                              |

```
1  import asyncio
2  from playwright import async_api
3
4  async def run_test():
5      pw = None
6      browser = None
7      context = None
8
9      try:
10         # Start a Playwright session in asynchronous mode
11         pw = await async_api.async_playwright().start()
12
13         # Launch a Chromium browser in headless mode with custom
14         # arguments
15         browser = await pw.chromium.launch(
16             headless=True,
17             args=[
18                 "--window-size=1280,720",          # Set the browser
19                 # window size
20                 "--disable-dev-shm-usage",          # Avoid using /dev/shm
21                 # which can cause issues in containers
22                 "--ipc=host",                       # Use host-level IPC
23                 # for better stability
24                 "--single-process"                   # Run the browser in a
25                 # single process mode
26             ],
27         )
28
29         # Create a new browser context (like an incognito window)
30         context = await browser.new_context()
31         context.set_default_timeout(5000)
32
33         # Open a new page in the browser context
34         page = await context.new_page()
35
36         # Navigate to your target URL and wait until the network request
37         # is committed
38         await page.goto("http://localhost:5173/", wait_until="commit",
39                         timeout=10000)
40
41         # Wait for the main page to reach DOMContentLoaded state
42         # (optional for stability)
43         try:
44             await page.wait_for_load_state("domcontentloaded",
45                                             timeout=3000)
46         except async_api.Error:
47             pass
48
49         # Iterate through all iframes and wait for them to load as well
50         for frame in page.frames:
51             try:
52                 await frame.wait_for_load_state("domcontentloaded",
53                                                 timeout=3000)
54             except async_api.Error:
55                 pass
56
57         # Interact with the page elements to simulate user flow
```



```
49         await asyncio.sleep(5)
50
51     finally:
52         if context:
53             await context.close()
54         if browser:
55             await browser.close()
56         if pw:
57             await pw.stop()
58
59 asyncio.run(run_test())
60
```

## Session expiry, token refresh, and graceful re-auth

### ATTRIBUTES

Status Failed

Priority Medium

Description Given an authenticated user with short-lived tokens, when the token expires during an active session, then the app transparently refreshes the token (if applicable) and retries the original request; if refresh fails, the user is redirected to login with preserved intent/URL. Validate UI during refresh and error flows. Precondition: adjustable token TTL in test environment. Clean-up: restore TTL.

Preview Link <https://testsprite-videos.s3.us-east-1.amazonaws.com/94380408-c041-70ff-803a-b96762bf1a56/1769087614311706//tmp/125cb02b-2813-4c4c-97ee-27202c94edbd/result.webm>

```
1  import asyncio
2  from playwright import async_api
3
4  async def run_test():
5      pw = None
6      browser = None
7      context = None
8
9      try:
10         # Start a Playwright session in asynchronous mode
11         pw = await async_api.async_playwright().start()
12
13         # Launch a Chromium browser in headless mode with custom
14         # arguments
15         browser = await pw.chromium.launch(
16             headless=True,
17             args=[
18                 "--window-size=1280,720",          # Set the browser
19                 # window size
20                 "--disable-dev-shm-usage",          # Avoid using /dev/shm
21                 # which can cause issues in containers
22                 "--ipc=host",                       # Use host-level IPC
23                 # for better stability
24                 "--single-process"                   # Run the browser in a
25                 # single process mode
26             ],
27         )
28
29         # Create a new browser context (like an incognito window)
30         context = await browser.new_context()
31         context.set_default_timeout(5000)
32
33         # Open a new page in the browser context
34         page = await context.new_page()
35
36         # Navigate to your target URL and wait until the network request
37         # is committed
38         await page.goto("http://localhost:5173/", wait_until="commit",
39                         timeout=10000)
40
41         # Wait for the main page to reach DOMContentLoaded state
42         # (optional for stability)
43         try:
44             await page.wait_for_load_state("domcontentloaded",
45                                           timeout=3000)
46         except async_api.Error:
47             pass
48
49         # Iterate through all iframes and wait for them to load as well
50         for frame in page.frames:
51             try:
52                 await frame.wait_for_load_state("domcontentloaded",
53                                                 timeout=3000)
54             except async_api.Error:
55                 pass
56
57         # Interact with the page elements to simulate user flow
```

```
49         await asyncio.sleep(5)
50
51     finally:
52         if context:
53             await context.close()
54         if browser:
55             await browser.close()
56         if pw:
57             await pw.stop()
58
59 asyncio.run(run_test())
60
```

## Form submission with client & server validation and optimistic UI

### ATTRIBUTES

Status Failed

Priority High

Description Given a user on a multi-field form (required fields, complex constraints), when the user submits valid data, then the form shows optimistic feedback, server persists data, and confirmation UI appears; when the user submits invalid data, then client-side validation prevents submission and shows errors; when server returns validation errors, then the form displays server messages inline. Precondition: API test sandbox. Clean-up: delete created records.

Preview Link <https://testsprite-videos.s3.us-east-1.amazonaws.com/94380408-c041-70ff-803a-b96762bf1a56/1769087614609851/tmp/8c72912d-1593-4421-8b04-c10b4c659de7/result.webm>

```
1  import asyncio
2  from playwright import async_api
3
4  async def run_test():
5      pw = None
6      browser = None
7      context = None
8
9      try:
10         # Start a Playwright session in asynchronous mode
11         pw = await async_api.async_playwright().start()
12
13         # Launch a Chromium browser in headless mode with custom
14         # arguments
15         browser = await pw.chromium.launch(
16             headless=True,
17             args=[
18                 "--window-size=1280,720",          # Set the browser
19                 # window size
20                 "--disable-dev-shm-usage",          # Avoid using /dev/shm
21                 # which can cause issues in containers
22                 "--ipc=host",                       # Use host-level IPC
23                 # for better stability
24                 "--single-process"                   # Run the browser in a
25                 # single process mode
26             ],
27         )
28
29         # Create a new browser context (like an incognito window)
30         context = await browser.new_context()
31         context.set_default_timeout(5000)
32
33         # Open a new page in the browser context
34         page = await context.new_page()
35
36         # Navigate to your target URL and wait until the network request
37         # is committed
38         await page.goto("http://localhost:5173/", wait_until="commit",
39                         timeout=10000)
40
41         # Wait for the main page to reach DOMContentLoaded state
42         # (optional for stability)
43         try:
44             await page.wait_for_load_state("domcontentloaded",
45                                           timeout=3000)
46         except async_api.Error:
47             pass
48
49         # Iterate through all iframes and wait for them to load as well
50         for frame in page.frames:
51             try:
52                 await frame.wait_for_load_state("domcontentloaded",
53                                                 timeout=3000)
54             except async_api.Error:
55                 pass
56
57         # Interact with the page elements to simulate user flow
58
```

```
49         await asyncio.sleep(5)
50
51     finally:
52         if context:
53             await context.close()
54         if browser:
55             await browser.close()
56         if pw:
57             await pw.stop()
58
59 asyncio.run(run_test())
60
```

## Offline behavior, local caching, and sync after reconnect

### ATTRIBUTES

Status Failed

Priority Medium

Description Given a user performs actions while offline (create/edit/delete), when connectivity is lost and then restored, then the app queues mutations, provides clear offline indicators, syncs with server on reconnect, resolves conflicts deterministically (or surfaces them), and shows final consistent state. Precondition: mock network control. Clean-up: reconcile test data.

Preview Link <https://testsprite-videos.s3.us-east-1.amazonaws.com/94380408-c041-70ff-803a-b96762bf1a56/1769087613777173//tmp/f5956db3-0906-4a82-bd60-2637b30e384e/result.webm>



```
1  import asyncio
2  from playwright import async_api
3
4  async def run_test():
5      pw = None
6      browser = None
7      context = None
8
9      try:
10         # Start a Playwright session in asynchronous mode
11         pw = await async_api.async_playwright().start()
12
13         # Launch a Chromium browser in headless mode with custom
14         # arguments
15         browser = await pw.chromium.launch(
16             headless=True,
17             args=[
18                 "--window-size=1280,720",          # Set the browser
19                 # window size
20                 "--disable-dev-shm-usage",          # Avoid using /dev/shm
21                 # which can cause issues in containers
22                 "--ipc=host",                       # Use host-level IPC
23                 # for better stability
24                 "--single-process"                   # Run the browser in a
25                 # single process mode
26             ],
27         )
28
29         # Create a new browser context (like an incognito window)
30         context = await browser.new_context()
31         context.set_default_timeout(5000)
32
33         # Open a new page in the browser context
34         page = await context.new_page()
35
36         # Navigate to your target URL and wait until the network request
37         # is committed
38         await page.goto("http://localhost:5173/", wait_until="commit",
39                         timeout=10000)
40
41         # Wait for the main page to reach DOMContentLoaded state
42         # (optional for stability)
43         try:
44             await page.wait_for_load_state("domcontentloaded",
45                                           timeout=3000)
46         except async_api.Error:
47             pass
48
49         # Iterate through all iframes and wait for them to load as well
50         for frame in page.frames:
51             try:
52                 await frame.wait_for_load_state("domcontentloaded",
53                                                 timeout=3000)
54             except async_api.Error:
55                 pass
56
57         # Interact with the page elements to simulate user flow
```

```
49         await asyncio.sleep(5)
50
51     finally:
52         if context:
53             await context.close()
54         if browser:
55             await browser.close()
56         if pw:
57             await pw.stop()
58
59     asyncio.run(run_test())
60
```

## File upload, preview, validation, and retry

### ATTRIBUTES

|              |                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|--------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Status       | Failed                                                                                                                                                                                                                                                                                                                                                                                                                             |
| Priority     | Medium                                                                                                                                                                                                                                                                                                                                                                                                                             |
| Description  | <p>Given a form that accepts file attachments, when the user selects valid and invalid files (size/type), then client-side validation and previews show, uploads are streamed to the server, progress is visible, retries on transient failures work, and final submission associates the uploaded file. Also test cancelling an upload. Precondition: test file fixtures and upload sandbox. Clean-up: remove uploaded files.</p> |
| Preview Link | <p><a href="https://testsprite-videos.s3.us-east-1.amazonaws.com/94380408-c041-70ff-803a-b96762bf1a56/1769087614230573/tmp/ae167d43-19e6-4011-bb92-51275e227d78/result.webm">https://testsprite-videos.s3.us-east-1.amazonaws.com/94380408-c041-70ff-803a-b96762bf1a56/1769087614230573/tmp/ae167d43-19e6-4011-bb92-51275e227d78/result.webm</a></p>                                                                               |

```
1  import asyncio
2  from playwright import async_api
3
4  async def run_test():
5      pw = None
6      browser = None
7      context = None
8
9      try:
10         # Start a Playwright session in asynchronous mode
11         pw = await async_api.async_playwright().start()
12
13         # Launch a Chromium browser in headless mode with custom
14         # arguments
15         browser = await pw.chromium.launch(
16             headless=True,
17             args=[
18                 "--window-size=1280,720",          # Set the browser
19                 # window size
20                 "--disable-dev-shm-usage",          # Avoid using /dev/shm
21                 # which can cause issues in containers
22                 "--ipc=host",                       # Use host-level IPC
23                 # for better stability
24                 "--single-process"                   # Run the browser in a
25                 # single process mode
26             ],
27         )
28
29         # Create a new browser context (like an incognito window)
30         context = await browser.new_context()
31         context.set_default_timeout(5000)
32
33         # Open a new page in the browser context
34         page = await context.new_page()
35
36         # Navigate to your target URL and wait until the network request
37         # is committed
38         await page.goto("http://localhost:5173/", wait_until="commit",
39             timeout=10000)
40
41         # Wait for the main page to reach DOMContentLoaded state
42         # (optional for stability)
43         try:
44             await page.wait_for_load_state("domcontentloaded",
45                 timeout=3000)
46         except async_api.Error:
47             pass
48
49         # Iterate through all iframes and wait for them to load as well
50         for frame in page.frames:
51             try:
52                 await frame.wait_for_load_state("domcontentloaded",
53                     timeout=3000)
54             except async_api.Error:
55                 pass
56
57         # Interact with the page elements to simulate user flow
58
```

```
49         await asyncio.sleep(5)
50
51     finally:
52         if context:
53             await context.close()
54         if browser:
55             await browser.close()
56         if pw:
57             await pw.stop()
58
59 asyncio.run(run_test())
60
```

## Email/password authentication: login, logout, and protected routes

### ATTRIBUTES

|              |                                                                                                                                                                                                                                                                                                                                                                                                                |
|--------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Status       | Failed                                                                                                                                                                                                                                                                                                                                                                                                         |
| Priority     | High                                                                                                                                                                                                                                                                                                                                                                                                           |
| Description  | Given a valid test account, when the user logs in with email/password, then they are authenticated, redirected to the dashboard, and tokens are stored; when the user logs out, then tokens are cleared and protected routes redirect to login. Also verify invalid credentials show correct errors and rate-limiting behavior. Precondition: test account present. Tear-down: reset account state if changed. |
| Preview Link | <a href="https://testsprite-videos.s3.us-east-1.amazonaws.com/94380408-c041-70ff-803a-b96762bf1a56/1769087613719461/tmp/d642dd62-3c35-4267-857f-ad4ae74c4628/result.webm">https://testsprite-videos.s3.us-east-1.amazonaws.com/94380408-c041-70ff-803a-b96762bf1a56/1769087613719461/tmp/d642dd62-3c35-4267-857f-ad4ae74c4628/result.webm</a>                                                                  |

```
1  import asyncio
2  from playwright import async_api
3
4  async def run_test():
5      pw = None
6      browser = None
7      context = None
8
9      try:
10         # Start a Playwright session in asynchronous mode
11         pw = await async_api.async_playwright().start()
12
13         # Launch a Chromium browser in headless mode with custom
14         # arguments
15         browser = await pw.chromium.launch(
16             headless=True,
17             args=[
18                 "--window-size=1280,720",          # Set the browser
19                 # window size
20                 "--disable-dev-shm-usage",          # Avoid using /dev/shm
21                 # which can cause issues in containers
22                 "--ipc=host",                       # Use host-level IPC
23                 # for better stability
24                 "--single-process"                   # Run the browser in a
25                 # single process mode
26             ],
27         )
28
29         # Create a new browser context (like an incognito window)
30         context = await browser.new_context()
31         context.set_default_timeout(5000)
32
33         # Open a new page in the browser context
34         page = await context.new_page()
35
36         # Navigate to your target URL and wait until the network request
37         # is committed
38         await page.goto("http://localhost:5173/", wait_until="commit",
39             timeout=10000)
40
41         # Wait for the main page to reach DOMContentLoaded state
42         # (optional for stability)
43         try:
44             await page.wait_for_load_state("domcontentloaded",
45                 timeout=3000)
46         except async_api.Error:
47             pass
48
49         # Iterate through all iframes and wait for them to load as well
50         for frame in page.frames:
51             try:
52                 await frame.wait_for_load_state("domcontentloaded",
53                     timeout=3000)
54             except async_api.Error:
55                 pass
56
57         # Interact with the page elements to simulate user flow
```

```
49         await asyncio.sleep(5)
50
51     finally:
52         if context:
53             await context.close()
54         if browser:
55             await browser.close()
56         if pw:
57             await pw.stop()
58
59 asyncio.run(run_test())
60
```



